

Roadmap visual y técnico

Tank Arena - transición hacia estilo Overgrown / Eco-Tech



Objetivo principal

Pasar de un prototipo funcional dibujado con Phaser.Graphics a una arena visualmente atractiva usando assets 2D top-down, sin perder la arquitectura correcta: el backend sigue siendo la verdad absoluta y el frontend solo renderiza el estado recibido.

Frontend	Phaser 3 + TypeScript + Socket.IO Client. Dibuja sprites, tiles, HUD, glows y efectos visuales.
Backend	NestJS + TypeScript + Socket.IO. Mantiene estado vivo de partida en memoria, calcula balas, colisiones, daño y mapa.
Base de datos	No usar para posiciones ni balas. Más adelante solo para usuarios, resultados, ranking y torneos.
Assets	PNG transparentes para obstáculos, tanques, tiles del suelo y efectos.

1. Decisión clave: separar lógica de visual

No necesitas cambiar la lógica del backend para mejorar el diseño. El backend puede seguir enviando obstáculos con x, y, width, height y type. El frontend simplemente decide qué sprite mostrar según el type.

```
// Backend envía esto
{
  id: 'bush_01',
  type: 'bush',
  x: 500,
  y: 300,
  width: 128,
  height: 96,
  hp: 1,
  destructible: true
}

// Frontend decide cómo se ve
spriteKey = 'bush_01';
```

La hitbox puede ser rectangular, pero el sprite puede verse orgánico, con hojas, flores o bordes irregulares. Esa separación es normal en videojuegos.

2. Orden recomendado de trabajo

Fase 1 - Obstáculos con assets	Reemplazar rectángulos por sprites para bush, wood, rock, steel y mirror. Mantener las mismas reglas del backend.
Fase 2 - Piso con tiles	Agregar tiles de piedra, grietas y musgo. Esto cambia mucho la percepción visual del juego.
Fase 3 - Tanques por partes	Crear tank_body y tank_turret por color. El cuerpo puede moverse y el cañón rotar hacia el mouse.
Fase 4 - Efectos	Agregar muzzle flash, explosiones, hojas al destruir bush, astillas para wood, polvo para rock y chispas cyan para mirror.
Fase 5 - Pulido visual	Iluminación, cámara, HUD, partículas, sonido, animación de espejo y feedback al recibir daño.

3. Assets mínimos para empezar

- bush_01.png - arbusto grande con hojas y flores, fondo transparente.
- wood_barricade_01.png - barricada de tablones, clavos y madera rota.
- rock_block_01.png - piedra antigua con grietas y musgo.
- steel_block_01.png - bloque metálico oscuro con placas y tornillos.
- mirror_panel_01.png - cristal/panel cyan para reflejar balas.
- floor_stone_01.png - tile base de piedra oscura.
- floor_moss_01.png - tile con musgo.
- floor_crack_01.png - tile con grieta.
- tank_body_green.png y tank_turret_green.png.
- tank_body_red.png y tank_turret_red.png.
- bullet_yellow.png, muzzle_flash.png, particle_glow.png.

4. Carpeta recomendada de assets

```
public/  
  assets/  
    tiles/  
      floor_stone_01.png  
      floor_stone_02.png  
      floor_moss_01.png  
      floor_crack_01.png  
    obstacles/  
      bush_01.png  
      bush_02.png  
      wood_barricade_01.png  
      rock_block_01.png  
      steel_block_01.png  
      mirror_panel_01.png  
    tanks/  
      tank_body_green.png  
      tank_turret_green.png  
      tank_body_red.png  
      tank_turret_red.png  
      tank_body_blue.png  
      tank_turret_blue.png  
    effects/  
      bullet_yellow.png  
      muzzle_flash.png  
      particle_glow.png  
      explosion_01.png
```

5. Cambios en Phaser

Actualmente usas Graphics para los obstáculos. Para assets conviene usar Containers, porque cada obstáculo puede tener sombra, sprite principal y glow.

```
private obstacleObjects: Map<string, Phaser.GameObjects.Container> = new Map();  
  
private createObstacleObject(obs: Obstacle): Phaser.GameObjects.Container {  
  const container = this.add.container(obs.x, obs.y).setDepth(2);  
  
  const shadow = this.add.ellipse(4, 6, obs.width * 0.9, obs.height * 0.7, 0x000000, 0.28);  
  const sprite = this.add.image(0, 0, this.getObstacleSpriteKey(obs));  
  
  sprite.setDisplaySize(obs.width, obs.height);  
  
  container.add([shadow, sprite]);  
  return container;  
}  
  
private getObstacleSpriteKey(obs: Obstacle): string {  
  const map: Record<ObstacleType, string> = {  
    bush: 'bush_01',  
    wood: 'wood_barricade_01',  
    rock: 'rock_block_01',  
    steel: 'steel_block_01',  
    mirror: 'mirror_panel_01',  
  };  
  
  return map[obs.type];  
}
```

6. Cargar assets en BootScene

```
preload(): void {  
    this.load.image('floor_stone_01', 'assets/tiles/floor_stone_01.png');  
    this.load.image('floor_moss_01', 'assets/tiles/floor_moss_01.png');  
    this.load.image('floor_crack_01', 'assets/tiles/floor_crack_01.png');  
  
    this.load.image('bush_01', 'assets/obstacles/bush_01.png');  
    this.load.image('wood_barricade_01', 'assets/obstacles/wood_barricade_01.png');  
    this.load.image('rock_block_01', 'assets/obstacles/rock_block_01.png');  
    this.load.image('steel_block_01', 'assets/obstacles/steel_block_01.png');  
    this.load.image('mirror_panel_01', 'assets/obstacles/mirror_panel_01.png');  
  
    this.load.image('tank_body_green', 'assets/tanks/tank_body_green.png');  
    this.load.image('tank_turret_green', 'assets/tanks/tank_turret_green.png');  
    this.load.image('bullet_yellow', 'assets/effects/bullet_yellow.png');  
}
```

7. Fondo tipo Overgrown Arena

El salto visual más grande vendrá del suelo. En vez de una cuadrícula plana, usa tiles de piedra oscura con variaciones. La cuadrícula puede quedar muy sutil o desaparecer.

```
private drawTileBackground(): void {
  const tileSize = 64;
  const keys = ['floor_stone_01', 'floor_stone_01', 'floor_moss_01', 'floor_crack_01'];

  for (let x = 0; x < this.mapW; x += tileSize) {
    for (let y = 0; y < this.mapH; y += tileSize) {
      const index = Math.abs((x * 31 + y * 17) % keys.length);
      const key = keys[index];
      this.add.image(x, y, key).setOrigin(0).setDepth(0);
    }
  }
}
```

Recomendación: si el mapa es grande, usa `TileSprite` o `Tilemap` más adelante. Para el MVP, imágenes repetidas están bien.

8. Backend: no cambia la autoridad

El backend no necesita conocer detalles visuales como flores, textura o brillo. Solo debe seguir enviando `type`, `posición`, `tamaño`, `hp` y reglas de colisión.

bush	1 disparo. Visual: vegetación con flores. Backend: hp 1, destructible true.
wood	2 disparos. Visual: barricada de madera. Backend: hp 2, destructible true.
rock	3 disparos. Visual: piedra con musgo. Backend: hp 3, destructible true.
steel	Indestructible. Visual: metal reforzado. Backend: destructible false.
mirror	Refleja balas. Visual: panel cyan. Backend: invierte <code>dirX</code> y <code>dirY</code> .

9. Prompt para crear assets

Puedes usar este prompt para generar un primer pack visual. Luego edítalo en Aseprite, Photoshop o Photopea para dejar fondos transparentes y tamaños consistentes.

```
Create a 2D top-down game asset pack for an eco-tech tank arena.

Style: dark overgrown ruins, mossy stone floor, futuristic cyan energy technology, readable top-down game assets, consistent scale, transparent background.

Create individual PNG assets:
1. bush obstacle, lush green leaves with small flowers
2. wooden barricade obstacle, planks and nails
3. cracked rock block with moss
4. futuristic dark steel block with bolts and blue details
5. cyan mirror energy panel for ricochet bullets
6. mossy stone floor tile
7. cracked stone floor tile
8. green top-down tank body
9. green top-down tank turret
10. glowing bullet projectile
11. muzzle flash
12. small explosion particles

Requirements: top-down view, transparent background for objects, consistent lighting, consistent scale, game-ready, readable silhouettes, not isometric, not side view, no text, no UI.
```

10. Checklist práctico

- Mantener backend autoritativo: cliente no decide daño, muerte, colisiones ni destrucción.
- Crear assets PNG transparentes para los 5 obstáculos.
- Cargar assets en BootScene o preload.
- Cambiar obsGfx por obstacleObjects con Containers.
- Dibujar obstáculos con sprites según obs.type.
- Usar setDisplaySize(obs.width, obs.height) para respetar tamaño del backend.
- Crear tiles de suelo y reemplazar la cuadrícula plana.
- Mantener Graphics para glows, HUD, barras de vida, balas y efectos simples.
- Separar tank body y tank turret cuando quieras mejorar el tanque.
- Agregar feedback visual al destruir obstáculos: hojas, polvo, astillas, chispas.

11. Qué no hacer todavía

- No metas Tiled todavía si tu backend ya genera el mapa.
- No metas spritesheets al inicio; usa PNG individuales.
- No cambies la lógica de colisión solo por cambiar visuales.
- No guardes posiciones o balas en PostgreSQL.
- No intentes hacer arte perfecto antes de que el gameplay esté estable.

Plan recomendado de 7 días

Día 1	Conseguir o generar los primeros 5 assets de obstáculos.
Día 2	Cargar assets en Phaser y reemplazar rectángulos por sprites.
Día 3	Crear tiles de suelo y fondo Overgrown.
Día 4	Mejorar espejo con glow y efecto de rebote visual.
Día 5	Separar tanque en cuerpo y cañón con sprites.
Día 6	Agregar efectos de destrucción por tipo de obstáculo.
Día 7	Pulir HUD, cámara, contraste y probar legibilidad en partida.

Resumen final

Tu código actual ya sirve como base. El siguiente salto no es reescribir la lógica, sino cambiar la capa visual: usar assets para obstáculos, tiles para el suelo y mantener Graphics para efectos. Esa es la forma más rápida de acercarte al estilo Overgrown Arena sin romper el backend autoritativo.